INTEGRACIÓN FRONTEND Y BACKEND

Gestión de estudiantes

Registro académico con React y Spring Boot

Objetivo

Implementar un directorio que permita registrar, buscar y eliminar estudiantes. La interfaz usa el estilo de navegación del Aula Virtual FISI como referencia.

Diseño de la solución

Estado y props	useState guarda la lista, el formulario y los filtros. App pasa datos y manejadores al directorio y al formulario mediante props.
Solicitudes	useReducer modela carga, éxito y error. React actualiza la lista tras la confirmación de la API. Spring Boot valida los datos y conserva el directorio en memoria.
Funciones adicionales	Edición de nombre y carrera con PUT, filtro combinado, orden, paginación, CSV de resultados y distribución por carrera.

Integrantes del grupo 7

Zafra Venegas, Andrey Hegel
Quispe Alvarado, Luis Guillermo
Ramirez Garcia, Estefano Alexis
Ramirez Alvarez, Maleck Adriano
Gutierrez Mamani, Joselin Victoria

Entrega y entorno

Las capturas muestran la aplicación conectada a Spring Boot en local. GitHub Pages ofrece el modo demostración con datos ficticios guardados en el navegador.

Aplicación en GitHub Pages · Código fuente y guía de ejecución

Directorio y búsqueda

El índice lateral, la cabecera y las pestañas siguen la referencia del Aula Virtual.

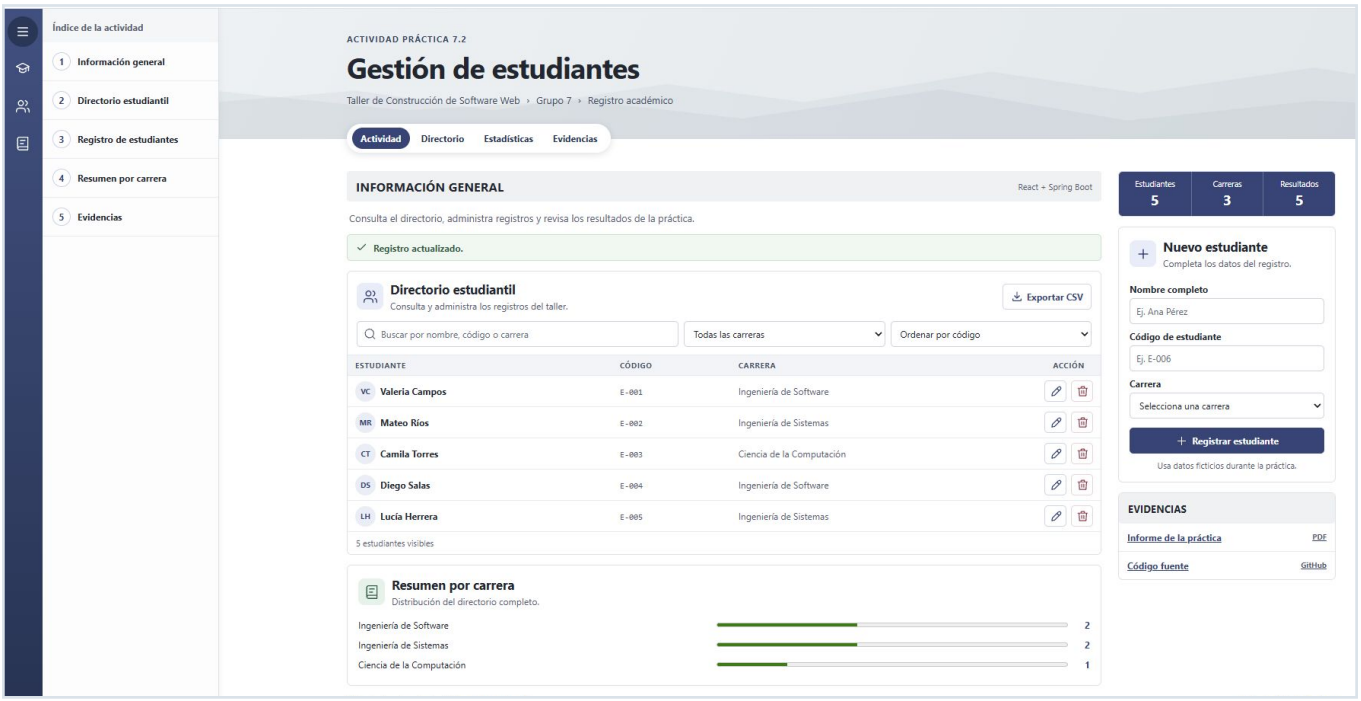


Figura 1. Directorio con la API conectada

El GET inicial devuelve cinco registros. El resumen muestra la distribución por carrera.



Figura 2. Búsqueda combinada

El texto «rios» y el filtro Ingeniería de Sistemas dejan a Mateo Ríos como único resultado.

Registro y actualización

El código identifica al estudiante y permanece fijo al editar su ficha.

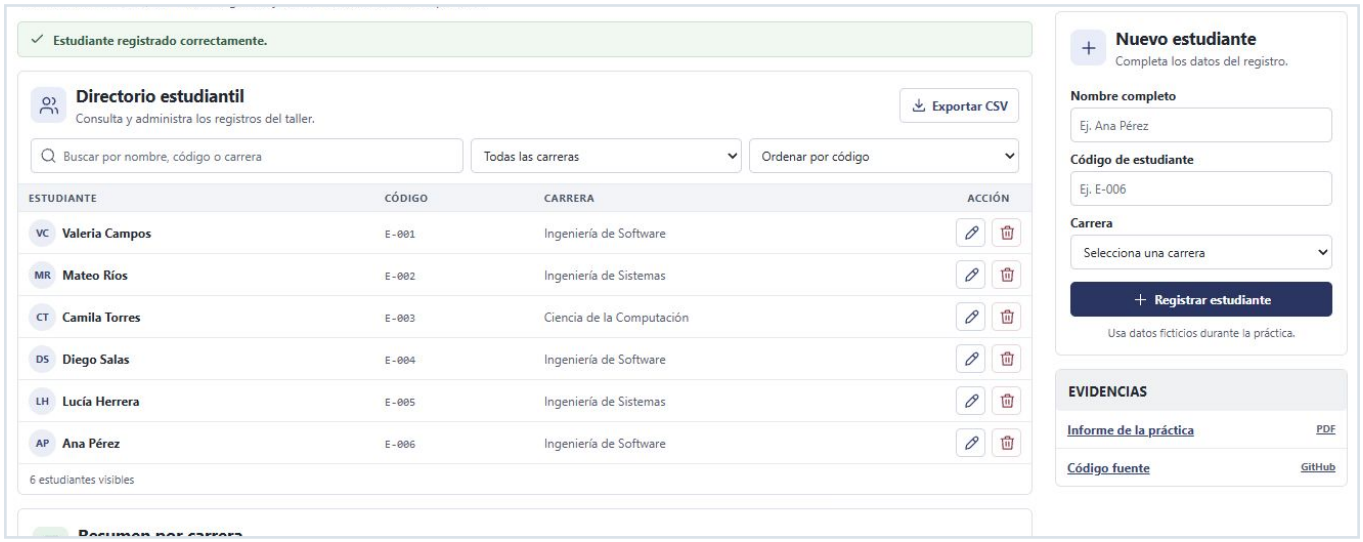


Figura 3. Alta confirmada

POST devuelve 201. Ana Pérez se incorpora con el código E-006.

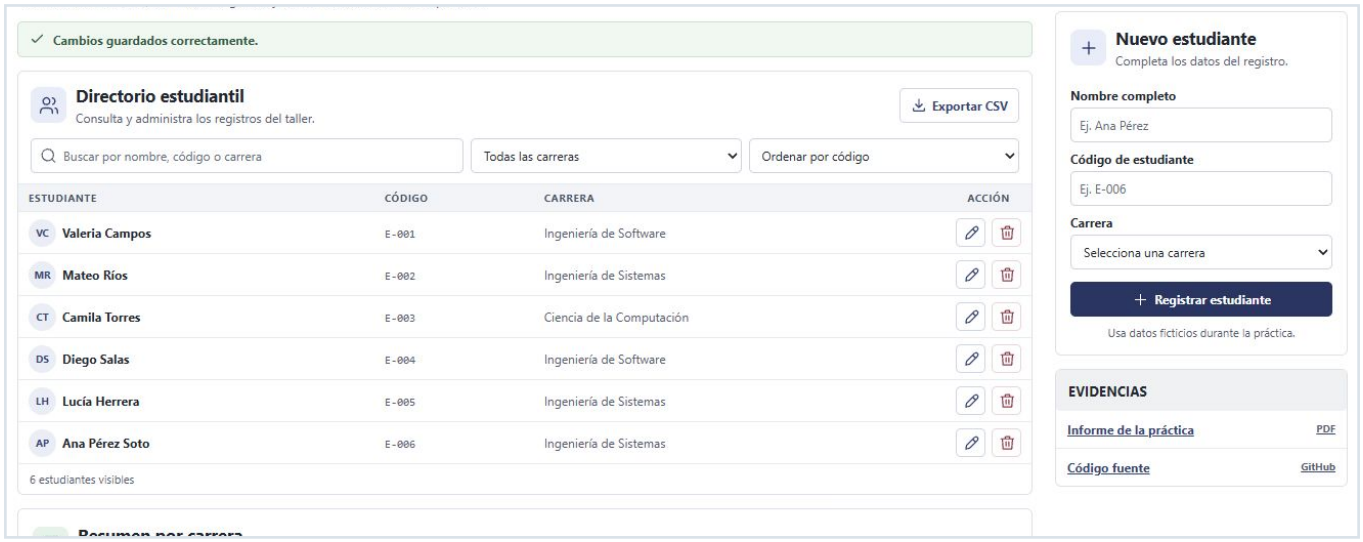


Figura 4. Edición confirmada

PUT devuelve 200. E-006 conserva el código, cambia a Ana Pérez Soto y pasa a Ingeniería de Sistemas.

Duplicados y eliminación

La interfaz muestra el resultado de la operación y conserva la lista ante un rechazo.

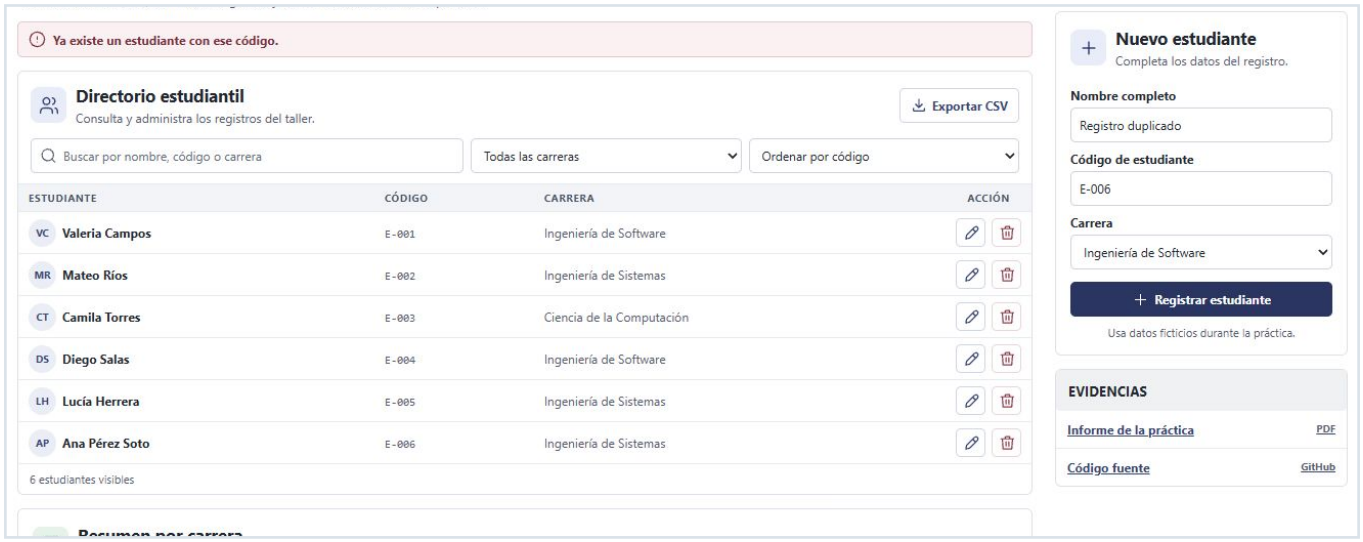


Figura 5. Código duplicado

POST devuelve 409. El intento de repetir E-006 mantiene el total de registros.

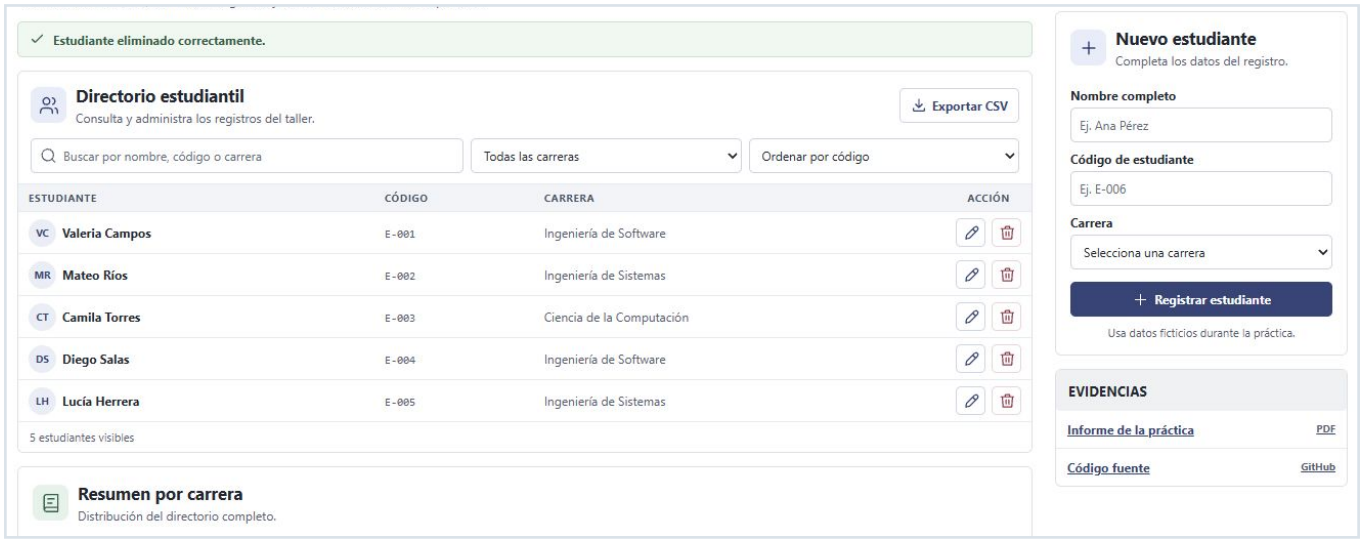


Figura 6. Baja confirmada

DELETE devuelve 204. Después de retirar los registros de prueba quedan cinco estudiantes.

Consulta y manejo de errores

La tabla muestra seis filas por página y el CSV incluye todo el resultado del filtro.

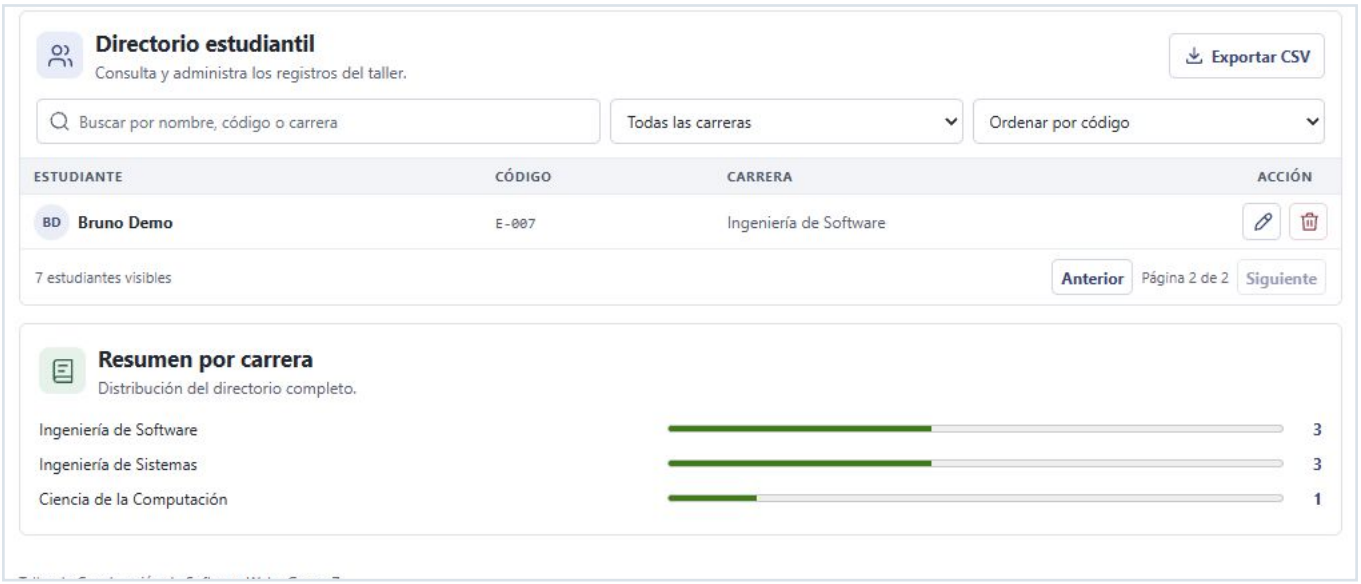


Figura 7. Segunda página del directorio
Con siete registros, la última fila pasa a la segunda página. El conteo mantiene el total filtrado.



Figura 8. Pérdida de conexión
Si la API se detiene, la vista muestra el error y ofrece Reintentar para recuperar el listado.

Comprobaciones adicionales

La API devuelve 400 ante campos inválidos o cambio de código, 404 si se intenta editar un registro inexistente y admite la solicitud CORS para PUT desde el origen de Pages.

Preguntas de evaluación

Conceptos de estado aplicados al registro de estudiantes.

1. ¿Qué es el estado y por qué importa?

Es la información que cambia durante la ejecución y determina qué muestra la interfaz. Al agregar un estudiante cambian la lista y el contador.

2. ¿En qué se diferencian props y state?

Las props son entradas que recibe un componente. El state es el estado que administra el componente y cuya actualización puede provocar un render.

3. ¿Para qué se usa useState?

Declara un valor de estado y devuelve una función para actualizarlo. Aquí guarda la lista, los campos del formulario y la búsqueda.

4. ¿Qué significa elevar el estado?

Mover el estado al ancestro común que lo necesita y compartir valores y manejadores por props. En esta práctica, App coordina el directorio y el formulario.

5. ¿En qué se diferencian Context API y Redux Toolkit?

Context distribuye valores en el árbol de componentes. Redux Toolkit organiza un almacén de estado con acciones, reducers y herramientas de depuración.

6. ¿Cuándo conviene usar useReducer?

Cuando varias acciones producen transiciones relacionadas. Aquí el reducer coordina el estado de carga, éxito y error al listar, agregar, editar y eliminar.

7. ¿En qué se diferencian el estado del cliente y el del servidor?

El cliente controla filtros y campos del formulario. El servidor administra los registros y React sincroniza una copia con las respuestas de la API.

Referencias técnicas

React · Manejo de estado
Redux Toolkit · Introducción
Spring · Servicio REST